

PROGRAMACIÓN DE COMUNICACIONES EN RED

Programación de Comunicaciones en Red

¿Qué vamos a ver en esta unidad?

- ➔ Conceptos básicos de comunicación, **protocolos** básicos de comunicación entre aplicaciones y modelos de computación distribuida
- ➔ Programación de aplicaciones con comunicación mediante **SOCKETS**
- ➔ Fundamentos del modelo **cliente/servidor**

Programación de Comunicaciones en Red

Conceptos previos.

Sistemas distribuidos: Con el aumento del uso de ordenadores y el auge de internet, ya no se trabaja de forma aislada, sino que múltiples ordenadores colaboran entre sí y se comunican a través de una red. Es lo que llamamos computación distribuida. Las **características fundamentales** de este modelo son:

- ⊕ Están formados por más de un elemento computacional distinto e independiente. (pueden ser un procesador dentro de una máquina, un ordenador en una red, una aplicación en internet... donde ninguno comparte memoria con el resto.)
- ⊕ Los elementos que forman el sistema distribuido no están sincronizados entre sí.
- ⊕ Todos los elementos están conectados a una red de comunicaciones.

Cloud computing es una forma de computación distribuida.

Un proveedor “abstrae” mecanismos internos para proporcionar servicios (dropbox, spotify...) y el usuario accede a ellos de forma transparente. A este proceso de abstracción se le conoce como “**virtualización de servicios**”

Programación de Comunicaciones en Red

Comunicación:

Emisor: Es quien emite el mensaje, PC o cualquier dispositivo informático.

Receptor: Es quien recibe la información.

*Ambos roles pueden intercambiarse de forma que el emisor pase a ser receptor y el receptor sea ahora el emisor.

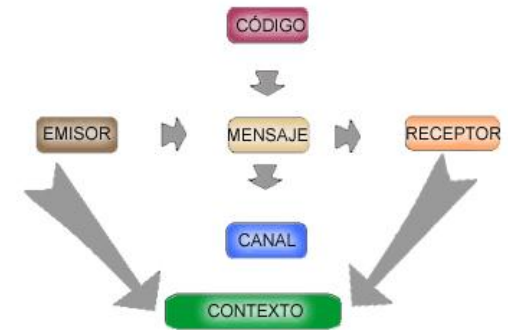
Canal: Es el medio físico por el que se transmite la información.

Mensaje: Información que se intercambia entre las aplicaciones que se comunican..

Paquete: El mensaje normalmente se divide en uno o más paquetes. Un paquete es la unidad básica de información que intercambian dos dispositivos de comunicación.

Protocolo: Normas mediante las cuales emisor y receptor se ponen de acuerdo, es decir, se deben marcar unas normas y procedimientos que regulan dicha comunicación.

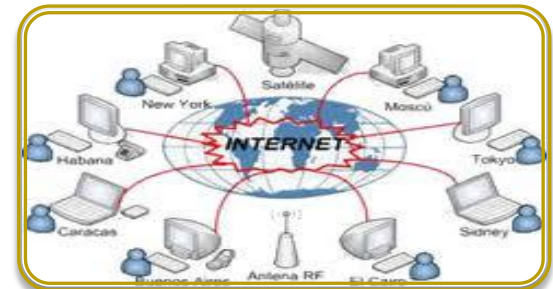
*Normas desde el tipo de cable usado, niveles de tensión que circulan por el mismo, hasta como se estructura la información dentro del mensaje (campos de la trama de información).



Programación de Comunicaciones en Red

Red:

Una red de ordenadores es un conjunto de equipos informáticos interconectados con el objeto de poder compartir información y recursos entre ellos.



LAN

- Red de área Local
- de reducida extensión,
- limitada usualmente a un único edificio o campus con un máximo de unos pocos kilómetros

MAN

- Red de área Metropolitana
- no supera una distancia de unas pocas decenas de kilómetros
- utilizadas para la interconexión de LAN ubicadas en diferentes recintos geográficos, como por ejemplo un campus universitario, un polígono industrial, etc.

WAN

- Red de área extensa
- se utilizan sobre espacios geográficos extensos
- Generalmente se utilizan para la transmisión de los datos los servicios de empresas portadoras

Programación de Comunicaciones en Red

Protocolos de comunicaciones:

El diseño de un sistema de comunicaciones es un problema complejo. Por esto, las redes se organizan en capas o niveles para reducir su complejidad (*divide y vencerás*).

Dentro de cada nivel de la arquitectura coexisten diferentes servicios. A la arquitectura por niveles también se le llama **jerarquía de protocolos**. Si los fabricantes quieren desarrollar productos compatibles, deberán ajustarse a los protocolos definidos para esa red.

En una jerarquía de protocolos se siguen las siguientes reglas:

- Cada nivel dispone de un conjunto de servicios.
- Los servicios están definidos mediante protocolos estándares.
- Cada nivel se comunica solamente con el nivel inmediato superior y con el inmediato inferior.
- Cada uno de los niveles inferiores proporciona servicios a su nivel superior.

Programación de Comunicaciones en Red

Modelos de referencia:

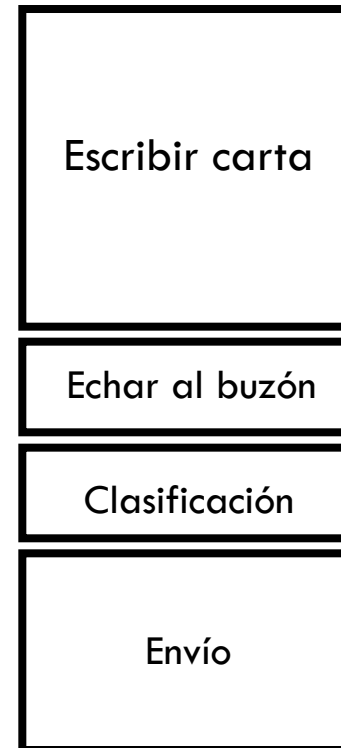
LA PILA OSI



LA PILA TCP/IP



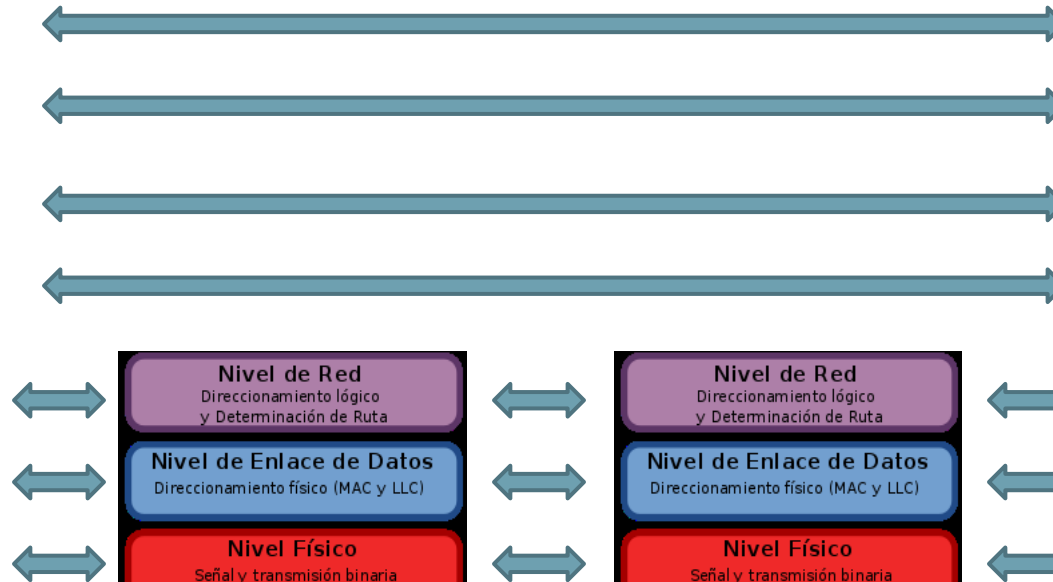
Modelo Correo Postal



Programación de Comunicaciones en Red

Modelo de referencia OSI:

EMISOR



RECEPTOR



NODOS INTERMEDIOS

Programación de Comunicaciones en Red

Modelo de referencia TCP/IP:

- **Capa host-red:** sólo se especifica que debe existir algún protocolo que conecte la estación (host) con la red. La razón fundamental, es que, TCP/IP se diseñó para que funcionase sobre redes diferentes, esta capa da independencia de la tecnología de la red. Ejemplos de protocolos tenemos SLIP, PPP, etc.
- **Capa de red (interred).** es la más importante de la arquitectura y su misión consiste en permitir que las estaciones envíen información (paquetes o tramas) a la red y los hagan viajar de forma independiente hacia su destino. El protocolo más importante de esta capa se llama **IP** (Internet Protocol o Protocolo de Interred).
- **Capa de transporte:** su función principal es establecer una conversación entre el origen y el destino, de igual forma que hace la capa de transporte en el modelo OSI. Los protocolos más destacados son el **TCP** (Transmisión Control Protocol) que ofrece un servicio fiable orientado a conexión, con lo que los paquetes llegan ordenados y sin errores. Y el **UDP** no orientado a la conexión y no fiable.
- **Capa de aplicación:** contiene todos los protocolos de alto nivel que se utilizan para ofrecer servicios a los usuarios. Podemos destacar, el terminal virtual (telnet), la transferencia de ficheros (ftp), el correo electrónico (smtp) y el servicio de nombres de dominio (dns), protocolo del World Wide web (**http y https**), etc...

Programación de Comunicaciones en Red

Protocolos orientados a conexión:

Un protocolo orientado a conexión es aquel en que el canal de comunicaciones entre dos aplicaciones permanece abierto durante cierto tiempo, permitiendo enviar múltiples mensajes de manera fiable.

Las operaciones que se llevan a cabo son:

- Establecimiento de la conexión
- Envío de mensajes
- Cierre de la conexión

El protocolo *NO orientado a conexión* es aquel en el que el canal de comunicación se crea de forma independiente para cada mensaje. No hay establecimiento ni cierre de la conexión.

Programación de Comunicaciones en Red

Protocolos de transporte:

TCP

- Protocolo de control de transmisión
- Es el protocolo de transporte más utilizado.
- Subdivide los mensajes del nivel de aplicación y crea paquetes que envía al nivel de Internet (y viceversa)
- Características:
 - Garantiza que los datos no se pierdan
 - Garantiza que los mensajes llegan en orden
 - Es un protocolo orientado a conexión

UDP

- Protocolo de datagrama de usuario
- Características:
 - No garantiza que los mensajes lleguen siempre
 - No garantiza que lleguen en orden
 - Permite enviar mensajes de 64KB máximo
 - Los mensajes se llaman datagramas
 - Es un protocolo no orientado a conexión, lo que lo hace más rápido que el TCP

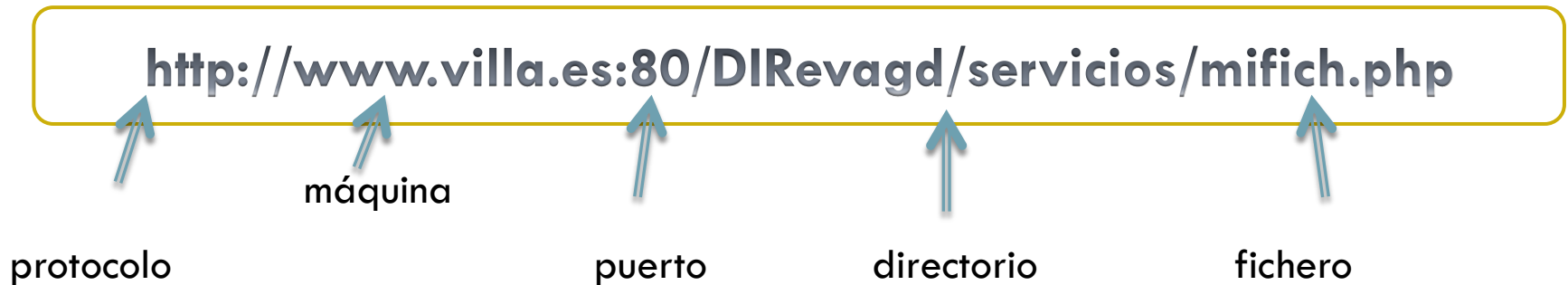
Programación de Comunicaciones en Red

Clase `InetAddress`: abstracción para las direcciones IP (*`java.net.Socket`*)

Método	Descripción
<code>InetAddress getLocalHost()</code>	Devuelve un objeto <code>InetAddress</code> que representa la dirección IP de la máquina donde se ejecuta el programa
<code>InetAddress getByName(String host)</code>	Devuelve un objeto <code>InetAddress</code> que representa la dirección IP de la máquina especificada como parámetro
<code>String getHostAddress()</code>	Devuelve la dirección IP en forma de cadena
<code>String getHostName()</code>	Devuelve el nombre del host

Programación de Comunicaciones en Red

Clase URL: representa un puntero a un recurso en la web (un fichero, un directorio, una consulta a una BD, etc...)



Programación de Comunicaciones en Red

Método	Descripción
<code>URL(String url)</code>	Uno de los constructores, crea un objeto URL
<code>URL(String protocolo, String host, int puerto, String fichero)</code>	Constructor, crea un objeto URL
<code>int getDefaultPort()</code>	Devuelve el puerto asociado por defecto al url
<code>int getPort()</code>	Devuelve el número de puerto de la URL
<code>String getHost()</code>	Devuelve el nombre de la máquina
<code>String getQuery()</code>	Devuelve la cadena que se envía a una máquina para ser procesada (lo que sigue al signo ?)
<code>String getUserInfo()</code>	Devuelve la parte con los datos del usuario de la dirección URL
<code>InputStream openStream()</code>	Abre una conexión al objeto URL, para la lectura de esa conexión
<code>URLConnection openConnection()</code>	Devuelve un objeto URLConnection que representa la conexión a un objeto remoto

Programación de Comunicaciones en Red

Clase `URLConnection`: clase abstracta que contiene métodos que permiten la comunicación entre la aplicación y una URL (leer y escribir).

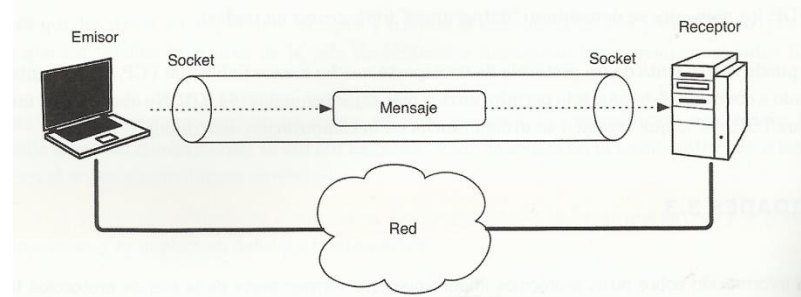
Método	Descripción
<code>InputStream getInputStream()</code>	Devuelve un objeto <code>InputStream</code> para leer datos de esta conexión
<code>OutputStream getOutputStream()</code>	Devuelve un objeto <code>OutputStream</code> para escribir datos en esta conexión
<code>void setDoInput(boolean b)</code>	Permite que el usuario reciba datos desde la URL si el parámetro <code>b</code> es <code>true</code> (por defecto es <code>true</code>)
<code>void setDoOutput(boolean b)</code>	Permite que el usuario envíe datos a la URL si el parámetro <code>b</code> es <code>true</code>
<code>void connect()</code>	Abre una conexión al recurso remoto si tal conexión no se ha establecido ya
<code>int getContentLength()</code>	Devuelve el valor del campo de cabecera <i>content-length</i> o <code>-1</code> si no está definido
<code>URL getURL()</code>	Devuelve la dirección URL

Programación de Comunicaciones en Red

SOCKETS.

Los sockets (“enchufes”) son un mecanismo de comunicación básico que se usa para realizar transferencias de información entre aplicaciones, bien en LAN o por internet. Proporcionan una abstracción de la pila de protocolos, ofreciendo una interfaz de programación sencilla con la que diferentes aplicaciones de un sistema distribuido pueden intercambiar mensajes.

Representa el extremo de un canal de comunicación establecido entre un emisor y un receptor. Para establecer una comunicación entre dos aplicaciones, **ambas** deben crear sus respectivos sockets y conectarlos entre sí. Una vez conectados, se crea un tubería privada a través de la red. Para enviar mensajes por el canal de comunicaciones, las aplicaciones *escriben* (*write*) en su socket, y para recibir mensajes *leen* (*read*) de su socket.



Programación de Comunicaciones en Red

Direcciones y puertos.

Cada máquina conectada a una red de comunicaciones se identifica por su **dirección IP** (IPv4 ó IPv6).

En IPv4, la dirección IP está formada por 32 bits (4 octetos): 10.0.1.25

Ahora bien, en una misma IP pueden existir múltiples aplicaciones que tienen comunicaciones con el exterior ¿cómo distinguirlas? Utilizando los **puertos**: número que identifica a un socket dentro de una máquina.

Cuando una aplicación crea un socket, debe especificar el puerto asociado a ese socket. El n° de puerto se representa con 16 bits (valores entre 0 y 65535)

Nunca puede haber más de un socket asignado a un mismo puerto en una máquina. Cuando una aplicación crea un socket y lo asigna a un puerto determinado, se dice que ese socket está *escuchando* (*listening*) por ese puerto.

Programación de Comunicaciones en Red

Tipos de sockets.

Dependiendo de su funcionalidad y del protocolo de nivel de transporte que utilizan, tenemos:

Sockets stream

- Son orientados a conexión (protocolo TCP)
- Son fiables y aseguran orden de entrega.
- Se utiliza para comunicarse siempre con el mismo receptor, manteniendo el canal de comunicación abierto entre ambas partes hasta que termina la conexión.

Sockets datagram

- No son orientados a conexión
- Se usa para enviar datagramas a multitud de receptores, ya que usan un canal temporal para cada envío.
- No son fiables, no aseguran orden de entrega correcto (protocolo UDP)

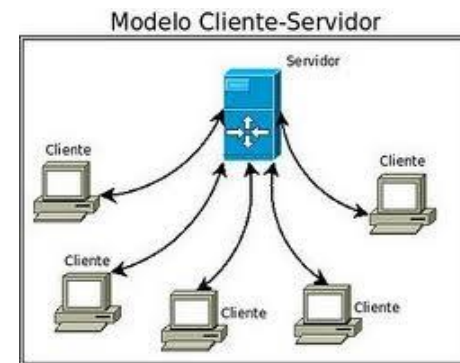
Programación de Comunicaciones en Red

Socket stream.

Cuando se establece una conexión usando socket stream (TCP), se debe seguir una secuencia de pasos. Uno de los elementos debe ejercer el papel de proceso servidor y otro el de proceso cliente.

El proceso servidor crea el socket en primer lugar y espera a que el cliente se conecte. Cuando el proceso cliente desea iniciar la comunicación, crea su socket y lo conecta al servidor creando así el canal de comunicación. En cualquier momento cualquiera de los dos puede cerrar su socket y finaliza la comunicación.

Los sockets TCP se utilizan en la gran mayoría de las aplicaciones IP. Algunos servicios con sus números e puerto reservados son: FTP (puerto 21), Telnet (23), HTTP (80), SMTP (25), HTTPS(443), ICQ (20000-20019), MSN Game Zone (28800 – 29000)



Programación de Comunicaciones en Red

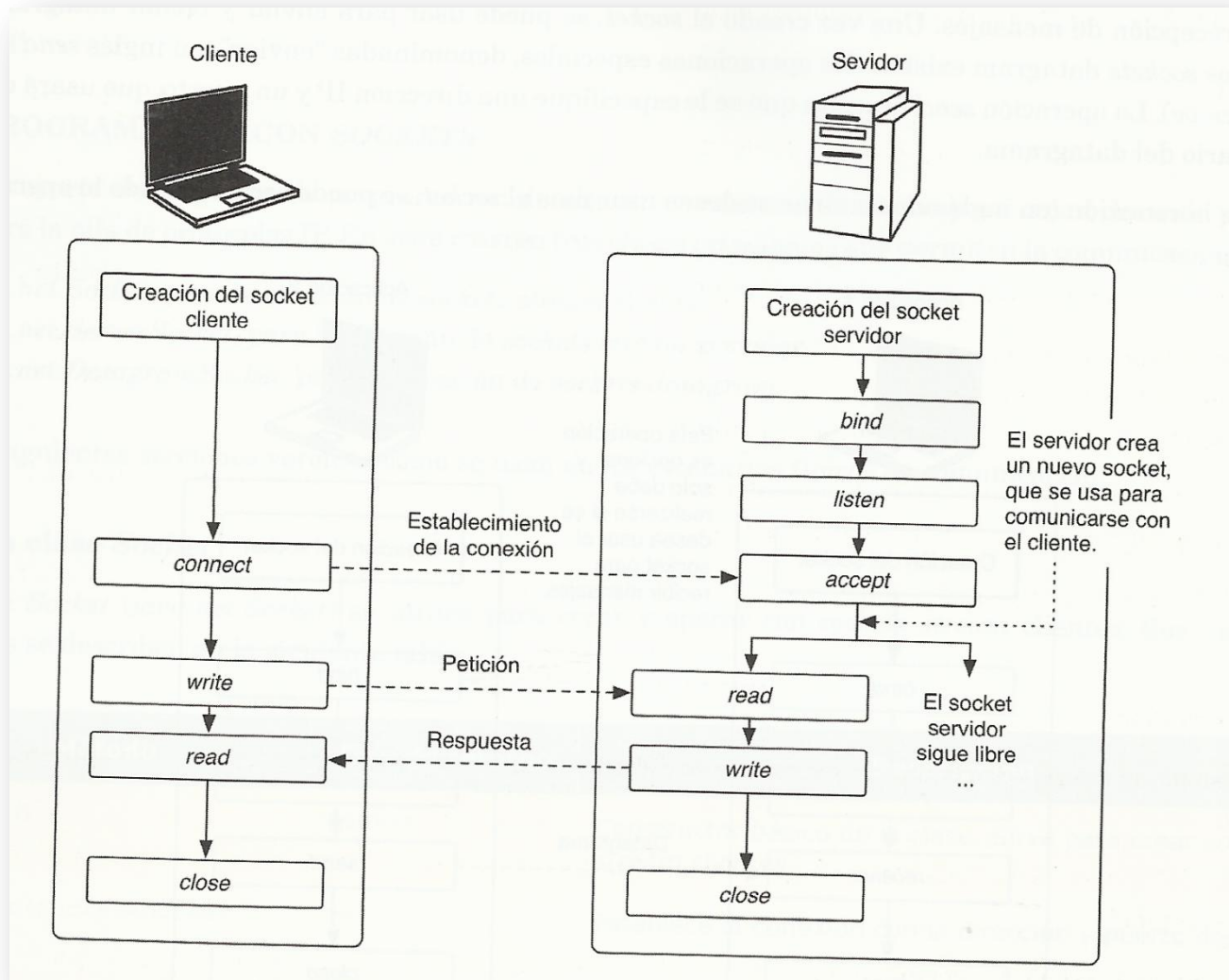
Proceso cliente.

- **Creación del socket.** El SO le asigna puerto. Socket cliente.
- **Conexión del socket** (connect). Localiza el socket del servidor y crea el canal de comunicación
- **Envío y recepción de mensajes.** Write y read sobre su socket cliente.
- **Cierre de la conexión** (close).

Proceso servidor.

- **Creación del socket.** Socket servidor.
- **Asignación de dirección y puerto** (bind).
- **Escucha** (listen). Se configura para escuchar por el puerto asignado y queda preparado para aceptar conexiones.
- **Aceptación de conexiones** (accept). Cuando llega una petición, se crea un nuevo socket dentro del proceso servidor, el socket servidor queda libre para seguir escuchando.
- **Envío y recepción de mensajes.** Write y read sobre el nuevo socket creado. EL socket servidor no se usa pues quedó fuera de la conexión.
- **Cierre de la conexión** (close). El socket servidor sigue estando disponible para nuevas conexiones

Programación de Comunicaciones en Red



Programación de Comunicaciones en Red

Sockets datagram.

Cuando se usa socket datagram no existe diferencia entre proceso servidor y proceso cliente. Todas las aplicaciones realizan los siguientes pasos:



Creación del socket.



Asignación de dirección y puerto (bind). Si se desea usar el socket para recibir mensajes hay que especificar IP y puerto.

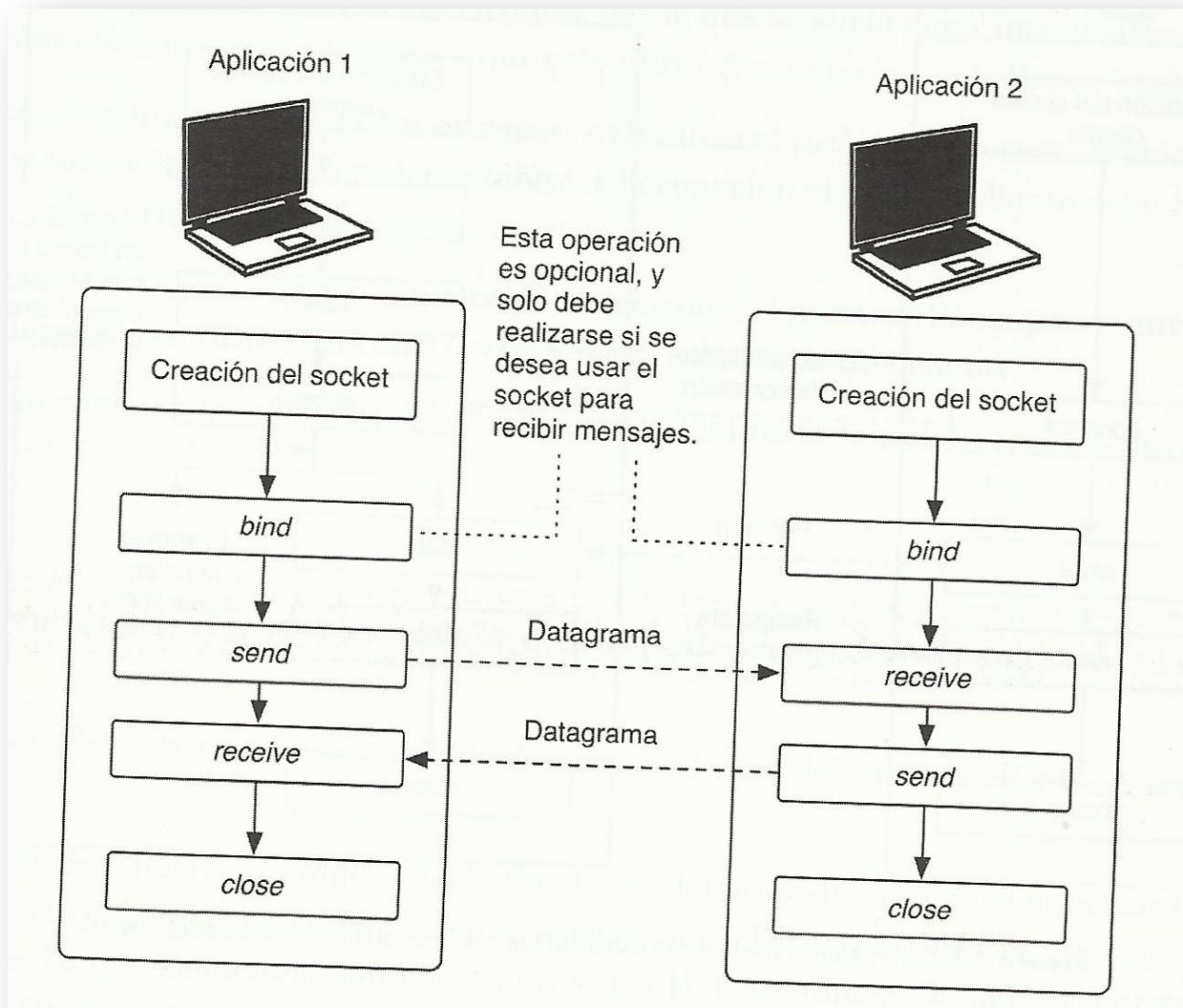


Envío y recepción de mensajes. Operaciones de enviar (**send**) y recibir (**receive**) datagramas. Send necesita que se especifique IP y puerto del destinatario



Cierre de la conexión (close). Si no se desea usar más el socket (como no hay canal, no se cierra canal de comunicación)

Programación de Comunicaciones en Red



Programación de Comunicaciones en Red

Clase *Socket*: para la creación de sockets stream cliente (java.net.Socket)

Método	Tipo retorno	Descripción
Socket()	Socket	Constructor básico de la clase. Crea sockets stream clientes
connect(SocketAddress addr)	void	Establece la conexión con la dirección y puerto de destino
getInputStream()	InputStream	Obtiene un objeto de clase InputStream que se usa para realizar operaciones de lectura (read)
getOutputStream()	OutputStream	Obtiene un objeto de clase OutputStream que se usa para realizar operaciones de escritura (write)
close()	void	Cierra el socket

Programación de Comunicaciones en Red

Clase `ServerSocket`: para la creación de sockets stream servidor ([java.net.ServerSocket](https://java.net/ServerSocket))

Método	Tipo retorno	Descripción
<code>ServerSocket()</code>	Socket	Constructor básico de la clase. Crea sockets stream servidores
<code>ServerSocket(String host, int port)</code>	Socket	Constructor alternativo de la clase. Realiza las operaciones de creación del socket y bind directamente
<code>bind(SocketAddress addr)</code>	void	Asigna el socket a una dirección y nº de puerto determinado (bind)
<code>accept()</code>	Socket	Escucha por el socket servidor (accept). Cuando llega una conexión devuelve un nuevo objeto Socket conectado al cliente.
<code>close()</code>	void	Cierra el socket

Programación de Comunicaciones en Red

Clase `DatagramSocket`: para la creación de sockets datagram (java.net.DatagramSocket)

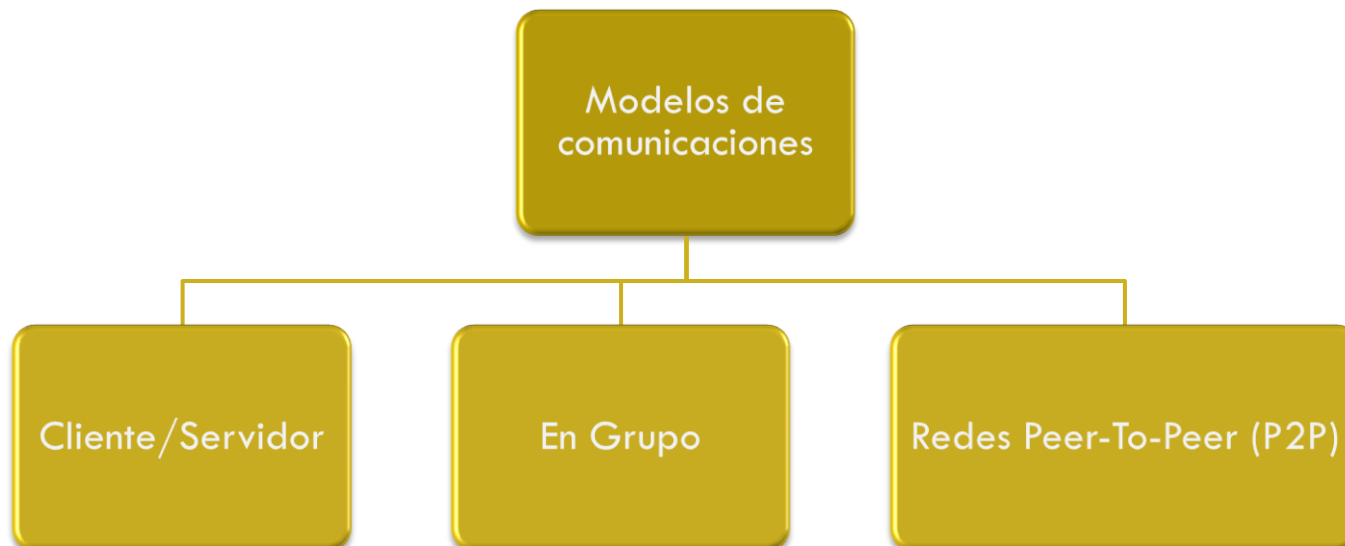
Método	Tipo retorno	Descripción
<code>DatagramSocket()</code>	<code>DatagramSocket</code>	Constructor básico de la clase. Crea sockets <i>datagram</i>
<code>DatagramSocket(SocketAddress bindaddr)</code>	<code>DatagramSocket</code>	Constructor alternativo de la clase. Realiza las operaciones de creación del socket y bind directamente
<code>send(DatagramPacket p)</code>	<code>void</code>	Envía un datagrama
<code>receive(DatagramPacket p)</code>	<code>void</code>	Recibe un datagrama
<code>close()</code>	<code>void</code>	Cierra el socket

Programación de Comunicaciones en Red

Modelos de comunicaciones.

Los sockets son herramientas básicas en la comunicación, permite enviar y recibir mensajes. Pero a la hora de desarrollar aplicaciones distribuidas hay que tener en cuenta aspectos de más alto nivel.

Un modelo de comunicaciones es una arquitectura general que especifica cómo se comunican entre sí los diferentes elementos de una aplicación distribuida: cuántos elementos tiene el sistema, qué función realiza cada uno...



Programación de Comunicaciones en Red

Modelo cliente/servidor.

En este modelo un proceso central, llamado servidor, ofrece una serie de servicios a uno o más procesos cliente. El proceso servidor debe estar alojado en una máquina fácilmente accesible en la red y conocida por los clientes.

Se basa en el mecanismo petición-respuesta.

En el modelo clásico, los roles de servidor y cliente son fijos. En los modelos más avanzados estos roles pueden cambiar según la situación.

**EL MODELO C/S ES EL MÁS UTILIZADO ACTUALMENTE
EN INTERNET. MÁS DEL 90% DE LAS APLICACIONES
DISTRIBUIDAS QUE USAMOS HABITUALMENTE UTILIZAN
ESTE MODELO.**



Programación de Comunicaciones en Red

Modelo cliente/servidor.

Es fácil ver que el modelo cliente-servidor se parece a los socket stream. Los roles son los mismos y el funcionamiento de los socket stream encaja con el modelo C/S.

Pero esto no quiere decir que el modelo cliente-servidor deba ser siempre implementado con socket stream. El modelo cliente-servidor es una arquitectura de tipo abstracto, que puede ser implementada utilizando diferentes tipos de mecanismos de comunicación, y los socket stream son uno de estos mecanismos, una alternativa más...

Cuando se elige un mecanismo de comunicación hay que tener en cuenta demás otros aspectos como la fiabilidad, la eficiencia...

Programación de Comunicaciones en Red

Modelo de comunicación en grupo.

En este modelo no existen roles diferenciados. En la comunicación en grupo existe un conjunto de dos o más elementos (procesos, aplicaciones...) que cooperan en un trabajo común. Al conjunto se le llama **grupo** y sus elementos se consideran iguales, sin roles ni jerarquías.

Los mensajes se transmiten mediante **radiado**, se envían de manera simultánea a todos los miembros, no se establecen comunicaciones punto-a-punto como en el modelo cliente/servidor.

Los últimos juegos on-line ya incorporan mecanismos de comunicación en grupo, para transmitir información entre todos los jugadores, aumentar la tolerancia a fallos y no saturar a los servidores, por ejemplo, el *Call of Duty*.

Programación de Comunicaciones en Red

Modelo de comunicación en grupo

El modelo en grupo se puede implementar con socket stream o datagram, dependiendo de las necesidades de la aplicación distribuida.

❖ Si se usa **socket stream**, cada elemento del grupo debe establecer una conexión con todas las demás. Desventaja: la creación de un gran número de sockets.

Ventaja: son fiables.

❖ Si se usan **datagram**, cada elemento del grupo necesita un único socket para comunicarse con todos los elementos del grupo. Desventaja: son menos fiables.

Ventaja: utiliza menor número de sockets.

Multicast

Se envía el mismo mensaje a varios destinatarios de forma simultánea

En IPv4, las IP van de 224.0.0.0 hasta 239.255.255.255 (clase D)

En IPv6 los dos primeros dígitos hexadecimales son FF

Ventaja: la comunicación es eficiente

Desventaja: riesgo para la seguridad y posibilidad de saturación

Programación de Comunicaciones en Red

Modelo de comunicación en grupo

La mayoría de aplicaciones distribuidas hacen uso de bibliotecas de comunicación, que son unas herramientas software específicamente diseñadas para implementar modelos de comunicación sofisticados, como la comunicación en grupo.

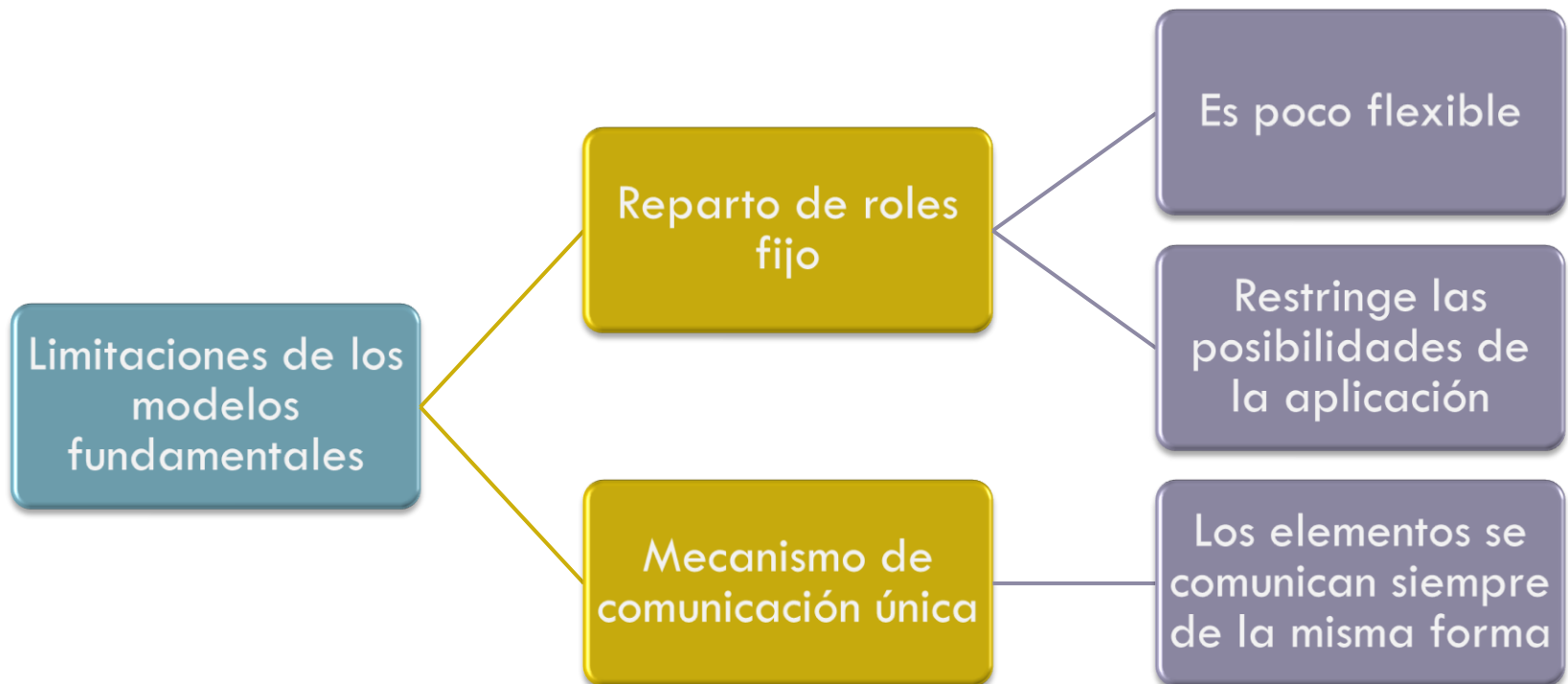
Un ejemplo típico es la biblioteca **JGroups**:

JGroups es una herramienta que soporta comunicación con grupos cerrados. Está escrita Java para explotar, en lo posible, todos los beneficios de este lenguaje. Al estar implementada en Java, puede ser considerado como una API en su totalidad, lo que le atribuye una enorme facilidad para su expansión e integración con otras herramientas para comunicación en grupo.

Programación de Comunicaciones en Red

Modelos híbridos y Redes P2P

En ocasiones, los modelos de comunicación reales implementados mezclan conceptos cliente-servidor y comunicación en grupo, dando lugar a enfoques híbridos. Los anteriores son potentes pero poco flexibles, con los modelos híbridos se intenta aprovechar las ventajas de cada uno.



Programación de Comunicaciones en Red

Modelos híbridos y Redes P2P

Las redes peer-to-peer son uno de los modelos de comunicación híbrida más potente (ej: Spotify, en sus inicios Napster para intercambio de archivos)

Una red P2P está formada por un grupo de elementos distribuidos que colaboran con un objetivo común. La diferencia es que en las redes P2P, cualquier elemento puede desempeñar los roles de cliente o servidor lo que le vuelve más potente.

Por ejemplo: una aplicación se conecta a internet como cliente, localiza un servidor y le hace una petición... con el tiempo, ese mismo cliente puede hacer a su vez de servidor para otros elementos de la red.

Ventajas del P2P:

El sistema es más tolerante a fallos al no existir un único servidor

Se reparte la carga de forma equilibrada entre todos los elementos y el sistema resulta más robusto.

